

กลุ่ม 7 — Iterative Structures

1. นายณฐภูมิ ปั่นหลวง — Concept + Short Code Illustration
2. นาย ปฏิภาณ นิลวงศ์ — Detailed Code + Live Demo
3. นาย ประเมษฐ์ ฟองดา — Rust vs Other Language + PPL Analysis
4. นาย ปิยวัฒน์ เตียนประไพ — Exercises + Common Mistakes + Challenge

Agenda — โครงสร้างการนำเสนอ (รวม ~20 นาที)

แต่ละคนนำเสนอประมาณ 5 นาที ต่อเนื่องกันตามลำดับ

1

Member 1

**Concept + Short Code
Illustration**

แนวคิดหลักของหัวข้อ พร้อมโค้ดตัวอย่างสั้น ๆ

~5 นาที

2

Member 2

Detailed Code + Live Demo

อธิบายโค้ดเชิงลึกและสาธิตการทำงานจริง

~5 นาที

3

Member 3

**Rust vs Other Language +
PPL Analysis**

เปรียบเทียบภาษา และวิเคราะห์เชิง
Programming Language

~5 นาที

4

Member 4

**Exercises + Common
Mistakes + Challenge**

ข้อผิดพลาดที่พบบ่อย และโจทย์ท้าทายผู้ฟัง

~5 นาที

Iterative Structures คืออะไร? (Concept)

- **Iterative Structures** คือโครงสร้างควบคุมที่ทำให้โปรแกรมทำคำสั่งเดิมซ้ำตามเงื่อนไขหรือข้อมูลที่กำหนด ใน Rust รูปแบบที่ใช้ทั่วไปคือ `loop`, `while`, `for` และ `while let` โดย `for` ทำงานร่วมกับ `iterator` เพื่อดึงข้อมูลที่ละค่า
- แนวคิดนี้มาจากความต้องการลดการเขียนคำสั่งซ้ำ ๆ เช่น แทนที่จะเขียน `println!` หลายครั้ง เราใช้ลูปเพื่อบอกโปรแกรมว่า “ทำคำสั่งนี้กับข้อมูลทุกตัว” หรือ “ทำซ้ำจนกว่าเงื่อนไขจะเป็นเท็จ”

ถ้าไม่มี `loop` เราต้องเขียนคำสั่งเดิมซ้ำ ซึ่งอาจทำให้โค้ดยาว แก่ไขยาก และไม่ยืดหยุ่น เช่น

```
fn main() {  
    println!("Hello");  
    println!("Hello");  
    println!("Hello");  
}
```

การนำโครงสร้างแบบ `Iterative Structure` มาใช้

```
fn main() {  
    for _ in 0..3 {  
        println!("Hello");  
    }  
}
```

Loop

- loop คือ การวนซ้ำแบบไม่กำหนดเงื่อนไขสิ้นสุดไว้ที่หัวลูป โปรแกรมจะทำงานต่อไปจนกว่าจะพบ break เพื่อให้สามารถกำหนดจุดสิ้นสุดของการวนซ้ำจากภายในกระบวนการทำงานได้อย่างชัดเจน แทนที่จะบังคับให้เงื่อนไขอยู่ที่หัว loop เสมอ นอกจากนี้ Rust อนุญาตให้ loop คำนวณผ่าน break value ได้ ทำให้ใช้เป็น expression ได้
- loop ช่วยในการแก้ไขปัญหา
 - งานที่ไม่ทราบจำนวนรอบล่วงหน้า
 - โปรแกรมที่ต้องรอเหตุการณ์หรือข้อมูล
 - ลูปที่มีเงื่อนไขหยุดหลายจุด
 - การเขียนลูปที่ต้องคืนค่าผลลัพธ์

Example

```
fn main() {
    let mut n = 1;

    loop {
        println!("{}", n);

        if n == 3 {
            break;
        }

        n += 1;
    }
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
1
2
3
```

While

- `while` คือการวนซ้ำที่ตรวจสอบเงื่อนไขก่อนทำงานแต่ละรอบ ถ้าเงื่อนไขเป็น `true` จึงทำงานต่อ แต่ถ้าเป็น `false` จะจบลง เหมาะกับสถานการณ์ที่ "เงื่อนไขเป็นตัวกำหนดว่าควรทำต่อหรือไม่" โดยตรง ทำให้โครงสร้างของโปรแกรมอ่านง่าย
- `while` ช่วยในการแก้ไขปัญห
 - การวนจนกว่าค่าจะถึงขีดจำกัด
 - การตรวจสอบสถานะซ้ำ ๆ
 - การทำงานที่จำนวนรอบขึ้นอยู่กับเงื่อนไข
 - ลดการเขียน `loop` ร่วมกับ `if` และ `break` ที่ซ้ำซ้อน

Example

```
fn main(){  
    let mut n = 1;  
  
    while n <= 3 {  
        println!("{}", n);  
        n += 1;  
    }  
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
1  
2  
3
```

For

- for คือโครงสร้างวนซ้ำที่ใช้สำหรับนำข้อมูลแต่ละตัวในชุดข้อมูล เช่น Array, Vector หรือ Range มาประมวลผลทีละตัว โดย Rust จะนำข้อมูลนั้นมาเป็น Iterator แล้วส่งค่าออกมาทีละตัวจนกว่าข้อมูลจะหมด จึงช่วยให้เราไม่ต้องจัดการจำนวนรอบหรือ index ด้วยตนเอง เพื่อให้สามารถกำหนดจุดสิ้นสุดของการวนซ้ำจากภายใน กระบวนการทำงานได้อย่างชัดเจน แทนที่จะบังคับให้เงื่อนไขอยู่ที่หัว loop เสมอ นอกจากนี้ Rust อนุญาตให้ loop คืบค่าผ่าน break value ได้ ทำให้ใช้เป็น expression ได้
- for ใช้แนวคิด iterator เพื่อให้การวนข้อมูล
 - อ่านง่าย
 - ทำงานกับ collection ได้อย่างเป็นระบบ
 - ลดการใช้ index ด้วยตนเอง
 - ลดความเสี่ยงจากการเข้าถึงตำแหน่งนอกขอบเขต
 - ทำงานร่วมกับ ownership และ borrowing ได้อย่างปลอดภัย

Example (Array)

```
fn main() {
    let numbers = [10, 20, 30];

    for number in numbers {
        println!("{}", number);
    }
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
10
20
30
```

Example (Range)

```
fn main() {
    for number in 1..=3 {
        println!("{}", number);
    }
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
1
2
3
```

Break

- `break` เป็นกลไกสำหรับ ยุติ `loop` ก่อนที่จะวนจนถึงจุดสิ้นสุดตามปกติ และสามารถระบุ `label` เพื่อออกจาก `loop` ชั้นนอกได้ โดย `break` สามารถใช้ได้กับ `loop`, `while` และ `for`
- ในบางครั้งโปรแกรมพบผลลัพธ์ที่ต้องการก่อนถึงเงื่อนไขปกติ Rust จึงนำหลักการของ `break` มาหยุดการทำงานของโปรแกรมทันทีแทนที่จะประมวลผลข้อมูลต่อโดยไม่จำเป็น เพื่อป้องกันการพบเงื่อนไขที่ผิดพลาด หยุดการค้นหาเมื่อพบข้อมูลที่ต้องการแล้ว และลดการทำงานที่ไม่จำเป็น

Example

```
fn main(){
    for n in 1..=10 {
        if n == 5 {
            break;
        }
        println!("{}",n);
    }
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
1
2
3
4
```

Continue

- Continue ใช้สำหรับ ข้ามการทำงานที่เหลือของรอบปัจจุบัน แล้วส่งการควบคุมกลับไปหัวของ loop เพื่อเริ่ม iteration ถัดไป เพื่อให้สามารถข้ามข้อมูลบางรายการโดยไม่ต้องหยุด loop ทั้งหมด
- continue ช่วยในการแก้ไขปัญห
 - ข้ามข้อมูลที่ไม่ต้องการประมวลผล
 - กรองค่าบางประเภท
 - ลดระดับการซ้อนของโค้ด
 - ทำให้เงื่อนไขผิดปกติดิบจบเร็ว

Example

```
fn main() {  
    for number in 1..=5 {  
        if number % 2 == 0 {  
            continue;  
        }  
        println!("{}", number);  
    }  
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
1  
3  
5
```

Nested Loops

- Nested Loop คือการเขียนลูปไว้ภายในลูปอีกชั้นหนึ่ง โดยทุกครั้งที่ outer loop ทำงาน 1 รอบ inner loop จะทำงานตามที่กำหนด นอกจากนี้ยังสามารถใช้ทั้ง break และ continue ได้ โดยถ้าไม่ระบุ label คำสั่งทั้งสองจะมีผลกับ ลูปชั้นในสุด ที่ครอบคำสั่งนั้นอยู่ หากต้องการควบคุมลูปชั้นนอก ให้ใช้ loop label เช่น 'outer'
- เหมาะกับข้อมูลสองมิติ เช่น ตาราง, กระดานเกม, matrix, แถว-คอลัมน์ หรือการเปรียบเทียบข้อมูลเป็นคู่

Example

```
fn main() {  
    for row in 1..=2 {  
        for column in 1..=3 {  
            println!("Row : {row}, Column : {column}");  
        }  
    }  
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
Row : 1, Column : 1  
Row : 1, Column : 2  
Row : 1, Column : 3  
Row : 2, Column : 1  
Row : 2, Column : 2  
Row : 2, Column : 3
```

Detailed Code + Live Demo

```
use std::io;
fn main() {
    loop {
        let mut input = String::new();
        println!("Enter a number (negative to stop):");

        io::stdin()
            .read_line(&mut input)
            .unwrap();

        let number: i32 = input.trim().parse().unwrap();

        if number < 0 {
            println!("Program stopped.");
            break;
        }

        println!("You entered: {}", number);
    }
}
```

อธิบายโปรแกรม

Run โปรแกรม และอธิบายการทำงานของ loop

ใส่ค่าบวก เช่น 10 → โปรแกรมแสดงผลและวนซ้ำต่อ

ใส่ค่าติดลบ เช่น -1 → break ทำงานและหยุดโปรแกรม

Detailed Code + Live Demo

```
use std::io;
fn main() {
    loop {
        let mut input = String::new();
        println!("Enter a number (negative to stop):");

        io::stdin()
            .read_line(&mut input)
            .unwrap();

        let number: i32 = input.trim().parse().unwrap();

        if number < 0 {
            println!("Program stopped.");
            break;
        }

        println!("You entered: {}", number);
    }
}
```

สิ่งที่ต้องระวังระหว่าง Demo

1. การใช้ `.unwrap()` กับ `.parse()`

ปัญหา: หากพิมพ์ตัวเลขแล้วกด enter
`.parse()` จะทำการคืน error ส่วน `.unwrap()`
จะทำการหยุดโปรแกรมทำให้เกิดอาการโปรแกรม panic

วิธีแก้: ใช้ `match` หรือ `if let` เพื่อป้องกัน
ข้อผิดพลาด

Rust vs [Other Language]

Aspect	Rust	[Other Language - Python]
Syntax	ใช้ loop, while, for ... in ... ไม่ต้องมีวงเล็บ () ล้อมรอบเงื่อนไข	ใช้ while, for ... in ... กำหนดบล็อกด้วย Indentation (การย่อหน้า) และมีโครงสร้างพิเศษอย่าง else ต่อท้ายลูป
Semantics / Behavior	loop เป็น Expression คืนค่าออกมาไว้ที่ตัวแปรได้ผ่าน break val;	ลูปเป็น Statement ไม่คืนค่าออกมาโดยตรง (ต้องใช้ List Comprehension หรือสร้างตัวแปรรับค่าเอง)
Type System	Static Typing — ตรวจสอบ Type ของ Iterator และข้อมูลตั้งแต่ช่วง Compile-time	Dynamic Typing / Duck Typing — ตรวจสอบการวนลูป (__iter__) ตอน Runtime
Memory Management	ควบคุมการเข้าถึงด้วย Ownership & Borrowing โดยไม่มี Garbage Collector (Zero-cost)	ใช้ Garbage Collector (GC) คอยจัดการ Memory มีภาระภายนอก (Runtime Overhead) ในทุกการวนลูป
Safety	Borrow Checker ป้องกันการแก้ไข Collection ขณะวนอ่านลูปตั้งแต่อยู่ในระดับ Compile-time	หากแก้ไข Collection ระหว่างวนลูป อาจเกิดพฤติกรรมผิดพลาด (Logical Bug) หรือแจ้งเตือน RuntimeError

Analysis: Python ออกแบบการวนลูปโดยเน้นความสะดวกสบายและอ่านง่ายของผู้เขียนโปรแกรม (Developer Ergonomics) ผ่าน Dynamic Typing แต่ต้องแลกมาด้วย Runtime Overhead จากการทำงานของ Garbage Collector และความเสี่ยงที่โปรแกรมจะทำงานผิดพลาดขณะรันหากมีการแก้ไขข้อมูลขณะวนลูป

ในทางกลับกัน ภาษา Rust เลือกใช้ระบบ **Borrow Checker** และ **Zero-cost Abstractions** ในการจัดการการวนซ้ำ ทำให้ได้ทั้งความปลอดภัยสูงสุดตั้งแต่ขั้นตอน Compile-time และประสิทธิภาพความเร็วที่เหนือกว่าโดยไม่ต้องพึ่งพา Garbage Collector

PPL Perspective — มุมมองเชิง Programming Language

Syntax

ใช้ Block Scoping {} ชัดเจน และตัดส่วนเกินออก

- ไม่บังคับใส่เครื่องหมายวงเล็บ () ล้อมรอบเงื่อนไข
- รองรับโครงสร้างหลัก loop, while, และ for .. in
- ช่วยลด Noise ในการเขียนโค้ดและอ่านง่ายขึ้น

Semantics

ยกระดับloopให้เป็น Expression-based Control Flow

- loop ใน Rust เป็น **Expression** (ต่างจาก Statement ทั่วไป)
- สามารถส่งคืนค่า (Return Value) ออกมาผ่านคำสั่ง break val;
- นำค่าที่ได้จากการวนloopไปกำหนดให้ตัวแปรได้โดยตรง

Type System

ควบคุมด้วย Static Typing และ Iterator Trait

- ใช้ **Iterator Trait** ในการนิยามพฤติกรรมการวนloop
- ตรวจสอบความถูกต้องของ Type และลำดับข้อมูลตั้งแต่ช่วง **Compile-time**
- ทำระบบ Type Inference ช่วยเดาชนิดข้อมูลโดยไม่ต้องประกาศชัดเจนทุกครั้ง

Memory / Resource

จัดการผ่าน Ownership และหลักการ RAII

- ควบคุมด้วยระบบ **Ownership & Borrowing** ป้องกัน Memory Leak
- ใช้แนวคิด **RAII** คืนทรัพยากรอัตโนมัติเมื่อหลุดออกจาก Scope
- ทำงานได้เร็วและสม่ำเสมอโดยไม่ต้องพึ่งพา **Garbage Collector**

Abstraction

Zero-cost Abstraction สไตล์ Functional

- รองรับ Functional Paradigm ผ่าน Iterator Combinators เช่น .map(), .filter()
- ซ่อนความซับซ้อนของการวนloopไว้ภายใต้ Abstraction ที่อ่านง่าย
- ได้ Performance ระดับล่าง (Low-level) โดยไม่มี Overhead เพิ่มเติม

Why Rust?

ป้องกันปัญหา Iterator Invalidation อย่างเด็ดขาด

- ระบบ **Borrow Checker** บล็อกการแก้ไขข้อมูลขณะวนloop ตั้งแต่ช่วง Compile-time
- ป้องกันบั๊กและข้อผิดพลาดร้ายแรงแบบ Runtime Error (เช่น ConcurrentModificationException)
- ได้ทั้ง ความปลอดภัย (Safety), ความน่าเชื่อถือ (Reliability) และ ความเร็วระดับ C/C++

Example— Counter-Controlled

โจทย์ (Problem)

เขียนลูปบวกค่าอาเรย์ตำแหน่ง 0-10 รวมตำแหน่ง 10 ด้วย

```
fn main(){
    let mut count = 0;
    let mut numbers =
Vec::new();
    for i in 0..=10 {
        count += i;
        numbers.push(count);
        println!("numbers = ",
numbers[i]);
    }
}
```

output

```
numbers = 0
numbers = 1
numbers = 3
numbers = 6
numbers = 10
numbers = 15
numbers = 21
numbers = 28
numbers = 36
numbers = 45
numbers = 55
```

Example— Logically-Controlled

โจทย์ (Problem)

จำลองโปรแกรม รับค่าจนกว่าจะได้เลขติดลบ และทำการหยุดดู

output

```
Enter a number (negative number to stop):
5
Input = 5
Enter a number (negative number to stop):
10
Input = 10
Enter a number (negative number to stop):
-1
Stopped! You entered negative number: -1
```

```
use std::io;
fn main(){
    let mut number: i32 = 0;
    while number >= 0 {
        let mut msg = String::new();
        println!("Enter a number (negative number to stop):");
        io::stdin()
            .read_line(&mut msg)
            .expect("Failed input");
        number = match msg.trim().parse() {
            Ok(num) => num,
            Err(_) => {
                println!("Please type a valid number!");
                0
            }
        };
        if number > 0{
            println!("Input = {}", number);
        }
    }
    println!("Stopped! You entered negative number: {}",
number);
}
```

Example— Iteration

โจทย์ (Problem)

เขียนโปรแกรมที่พิมพ์ "Name: Bob", "Name: Carol", "Name: Ted" จากคอลเลกชันของสตริงสามตัว

output

```
Names --> Bob
Names --> Carol
Names --> Ted
```

```
fn main(){
    let names = vec!["Bob", "Carol","Ted"];

    for name in names{
        println!("Names --> {}", name);
    }
}
```

Example— Nested Loop

โจทย์ (Problem)

เขียนโปรแกรมแสดงผล ตารางตัวเลขรูปสี่เหลี่ยมผืนผ้า ขนาด 3 X 4 โดยแต่ละช่องในตารางจะแสดงผลลัพธ์ของการนำ เลขแถวคูณกับเลขคอลัมน์

output

```
1 2 3 4
2 4 6 8
3 6 9 12
```

```
fn main(){
    let row = 3;
    let column = 4;
    for i in 1..=row{
        for j in 1..=column{
            print!("{}", i * j);
        }
        println();
    }
}
```

Common Mistakes — ข้อผิดพลาดที่พบบ่อย

✗ Incorrect

```
fn main() {  
    let mut count = 0;  
    let mut numbers = Vec::new();  
    for i in 0..=10 {  
        count += i;  
        numbers.push(count);  
        println!("numbers = ", numbers[i]);  
    }  
}
```

✓ Correct

```
fn main() {  
    let mut count = 0;  
    let mut numbers = Vec::new();  
    for i in 0..=10 {  
        count += i;  
        numbers.push(count);  
        println!("numbers = {}", numbers[i]);  
    }  
}
```

ทำไมถึงผิด? (Why?)

เกิดจากการสับสนเรื่องของ `format` ในการป้อนค่าในตัวแปร ออกมา

Common Mistakes — ข้อผิดพลาดที่พบบ่อย

✗ Incorrect

```
fn main() {  
    for i in (10..=0){  
        println!("i = {}", i);  
    }  
    println!("=====");  
    for i in (2=..20){  
        println!("i = {}", i);  
    }  
}
```

✓ Correct

```
fn main(){  
    for i in (0..=10).rev(){  
        println!("i = {}", i);  
    }  
    println!("=====");  
    for i in (0..=20).step_by(2){  
        println!("i = {}", i);  
    }  
}
```

ทำไมถึงผิด? (Why?)

ใน Rust ตัวดำเนินการ Range แบบนับถอยหลังโดยตรงด้วย `10..=0` จะไม่ทำงานเพราะ Range ใน Rust โดยเริ่มต้นจะสมมติว่าค่าเริ่มต้นต้อง น้อยกว่าหรือเท่ากับ ค่าสุดท้ายเสมอ

Challenge — คำถามท้าทายผู้ฟัง

โจทย์ (Problem)

สร้างตารางสูตรคูณแม่ 1 ถึง 9 คูณ 1 ถึง 9

ข้อกำหนด

- ข้ามการแสดงผลลัพธ์ที่เป็นจำนวนคี่
- หยุดพิมพ์ทั้งตารางทันทีเมื่อพบผลคูณที่มากกว่า 50 เป็นครั้งแรก
- นับว่าพิมพ์ผลคูณไปทั้งหมดกี่ค่า และแสดงผลรวมของค่าที่พิมพ์



Hint

1. ต้องใช้ for ซ้อนกันสองชั้น ช่วง 1 ถึง 9 ทั้งคู่
2. ลำดับการเช็คในลูปในสำคัญมาก ลองคิดว่าควรเช็ค "มากกว่า 50" ก่อนหรือหลังเช็ค "เป็นเลขคี่"
3. ผลคูณที่ทำให้หยุดจะไม่ถูกพิมพ์หรือนับ ตรวจให้แน่ใจว่าตรงกับที่โจทย์ต้องการ

```
fn main() {  
    let mut count = 0;  
    let mut sum = 0;  
    'outer: for i in 1..=9 {  
        for j in 1..=9 {  
            let product = i * j;  
  
            if product > 50 {  
                break 'outer;  
            }  
            if product % 2 != 0 {  
                continue;  
            }  
            println!("{}", i * j = {}, i, j, product);  
            count += 1;  
            sum += product;  
        }  
    }  
    println!("พิมพ์ทั้งหมด {} ค่า, ผลรวม = {}", count, sum);  
}
```

Challenge — คำถามท้าทายผู้ฟัง

โจทย์ (Problem)

โปรแกรมสุ่มเลขลับโดยที่ผู้เล่นทายจากชุดคำตอบที่กำหนดไว้ล่วงหน้า (50, 25, 40, 37)

ข้อกำหนด

1. ทุกครั้งที่ทาย ต้องบอกว่า "มากเกินไป" "น้อยไป" หรือ "ถูกต้อง"
2. ต้องทายอย่างน้อยหนึ่งครั้งเสมอ
3. จำกัดจำนวนครั้งที่ทายได้สูงสุด 5 ครั้ง
4. เมื่อจบเกม ให้แสดงว่าชนะหรือแพ้ และใช้ไปกี่ครั้ง



Hint

- โจทย์ต้องการ "ทำก่อน แล้วค่อยตรวจเงื่อนไข"
- เงื่อนไขหยุดมี 2 อย่าง คือ "ทายถูก" หรือ "ครบ 5 ครั้ง" ต้องเช็คทั้งคู่
- ระวังกรณีชุดคำตอบสั้นกว่า 5 ตัว ถ้าดึงเกินขอบเขตโปรแกรมจะ panic

Challenge — เฉลย

```
fn main() {
    let secret = 37;
    let guesses = [50, 25, 40, 37];
    let max_tries = 5;
    let limit = max_tries.min(guesses.len());
    let mut tries = 0;
    let mut won = false;
    loop {
        let guess = guesses[tries];
        tries += 1;
        match guess.cmp(&secret) {
            Ordering::Greater => println!("ครั้งที่ {}: ทาย {} -> มากไป", tries, guess),
            Ordering::Less => println!("ครั้งที่ {}: ทาย {} -> น้อยไป", tries, guess),
            Ordering::Equal => {
                println!("ครั้งที่ {}: ทาย {} -> ถูกต้อง!", tries, guess);
                won = true;
            }
        }
        if won || tries >= limit {
            break;
        }
    }
    if won {
        println!("ชนะ! ใช้ {} ครั้ง", tries);
    } else {
        println!("แพ้! ใช้ครบ {} ครั้งแล้ว", tries);
    }
}
```

Key Takeaways — สรุปสิ่งที่ควรจำ

- 1 loop ใน Rust ไม่ใช่แค่การวนลูป แต่คืนค่าได้
- 2 วนลูปอย่างปลอดภัยด้วย Ownership & Borrowing
- 3 เขียนง่ายระดับภาษาสูง แต่เร็วระดับภาษาเครื่อง
- 4 Iterative Structures แสดงให้เห็นถึงการทำงานร่วมกันของ **Type System**, **Memory Managemen** และ **Control Flow** ที่ออกแบบมาเพื่อขจัดข้อผิดพลาดอย่าง Out-of-bounds Access ได้อย่างเด็ดขาด

References — เอกสารอ้างอิง

- The Rust Programming Language (The Rust Book) — <https://doc.rust-lang.org/book/>
- Rust by Example — <https://doc.rust-lang.org/rust-by-example/>
- https://doc.rust-lang.org/rust-by-example/flow_control/loop.html
- <https://medium.com/@fennsaji/day-1-input-and-output-i-o-in-rust-with-examples-be6f9478d133>
- https://doc.rust-lang.org/rust-by-example/flow_control/for.html
- <https://doc.rust-lang.org/std/error/trait.Error.html#error-source>
- <https://doc.rust-lang.org/reference/expressions/loop-expr.html>
- <https://doc.rust-lang.org/book/ch03-05-control-flow.html>
- <https://mitaa.github.io/rust/doc/book/loops.html>

GitHub Repository: <https://github.com/670710136/Iterative-Structures>

Questions & Answers

กลุ่ม 7 — Iterative Structure